ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

OpenAI gym environment.

Gym is a standard API for reinforcement learning, and a diverse collection of reference environments.

Introduction: Basic Usage

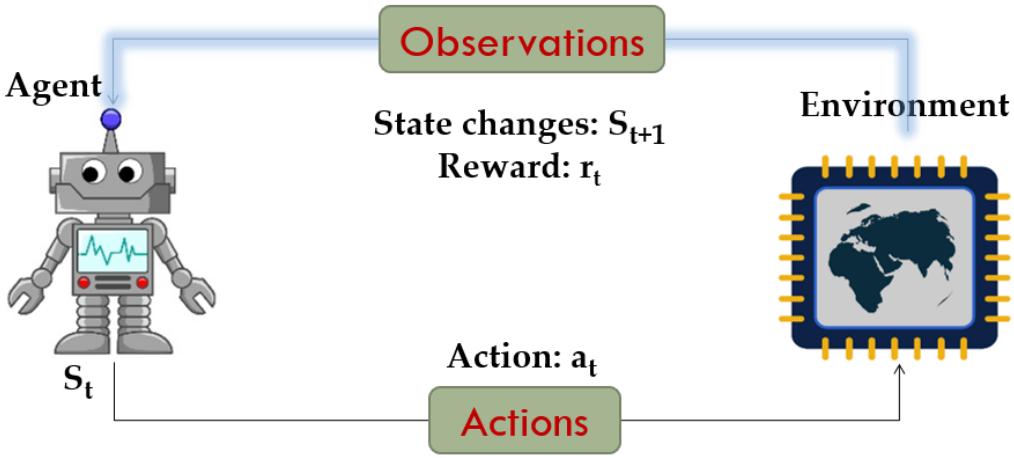
Initializing Environments

Initializing environments can be done via:

```
import gym
env = gym.make(' FrozenLake-v1')
```

Interacting with the Environment

Gym implements the classic "agent-environment loop":



The agent executes some actions in the environment and observes how the environment's state changes/responses. One such action-observation exchange is referred to as a *timestep*.
The goal in RL is to manipulate the environment in some specific way to satisfy a desired behavior. For example, train a car to drive itself without any human interaction, or train a bot to execute a trading order in a stock/financial markets. If it succeeds in doing this (or makes some progress towards that goal), it will receive a positive reward alongside the observation for this timestep. The reward may also be negative, if the agent did not yet succeed (or did not make any progress). The agent will then be trained to maximize the reward it accumulates over many timesteps.
After some timesteps, the environment may enter a terminal state. For instance, the robot may have crashed! In that case, we want to reset the environment to a new initial state. The environment issues a done signal to the agent if it enters such a terminal state. Not all done signals must be triggered

by a "catastrophic failure": Sometimes we also want to issue a done signal after a fixed number of timesteps, or if the agent has succeeded in completing some tasks in the environment.
Let's see what the agent-environment loop looks like in Gym. This example will run an instance of FrozenLake-v1 environment for 1000 timesteps. Since we pass render_mode="human", you should see a window pop up rendering the environment.

```
gym.make('MountainCar-v0', render_mode = "human")
```

Try to execute this example ion your machine:

```
import random
import gym

env =  gym.make('MountainCar-v0', render_mode = "human")

states = env.observation_space.shape[0]
actions = env.action_space.n

episodes = 10
for episode in range(1, episodes):
    state = env.reset()
    done = False
    score = 0
    while not done:
        action = random.choice([0,1])
        _, reward, done, _ , _ = env.step(action)
        score += reward
        env.render()
    print(f"Episode {episode} , Score: {score} ")

env.close()
```

Note: you Have to install the following packages: tensorflow==2.1.0 , keras-rl, gym

1. Core

a. gym.Env

gym.Env.**step(self, action: ActType) → Tuple[ObsType, float, bool, bool, dict]**

Run one timestep of the environment's dynamics.

When end of episode is reached, you are responsible for calling reset() to reset this environment's state. Accepts an action and returns a tuple *(observation, reward, terminated, truncated, info)*.

PARAMETERS:

action (ActType) – an action provided by the agent.

RETURNS:

observation (object) – this will be an element of the environment's observation_space. This may, for instance, be a numpy array containing the positions and velocities of certain objects.

reward (float) – The amount of reward returned as a result of taking the action.

terminated (bool) – whether a terminal state (as defined under the MDP of the task) is reached. In this case further step() calls could return undefined results.

truncated (bool) – whether a truncation condition outside the scope of the MDP is satisfied. Typically, a timelimit, but could also be used to indicate agent physically going out of bounds. Can be used to end the episode prematurely before a terminal state is reached.

info (dictionary) – info contains auxiliary diagnostic information (helpful for debugging, learning, and logging). This might, for instance, contain: metrics that describe the agent's performance state, variables that are hidden from observations, or individual reward terms that are combined to produce the total reward. It also can contain information that distinguishes truncation and termination; however, this is deprecated in favour of returning two booleans, and will be removed in a future version.

(deprecated)

done (bool) – A boolean value for if the episode has ended, in which case further step() calls will return undefined results. A done signal may be emitted for different reasons: Maybe the task underlying the environment was solved successfully, a certain timelimit was exceeded, or the physics simulation has entered an invalid state.

gym.Env.reset(self, *, seed: int | None = None, options: dict | None = None) → Tuple[ObsType, dict]

Resets the environment to an initial state and returns the initial observation.

This method can reset the environment's random number generator(s) if seed is an integer or if the environment has not yet initialized a random number generator. If the environment already has a random number generator and reset() is called with seed=None, the RNG should not be reset. Moreover, reset() should (in the typical use case) be called with an integer seed right after initialization and then never again.

PARAMETERS:

seed (*optional int*) – The seed that is used to initialize the environment's PRNG. If the environment does not already have a PRNG and seed=None (the default option) is passed, a seed will be chosen from some source of entropy (e.g. timestamp or /dev/urandom). However, if the environment already has a PRNG and seed=None is passed, the PRNG will *not* be reset. If you pass an integer, the PRNG will be reset even if it already exists. Usually, you want to pass an integer *right after the environment has been initialized and then never again*. Please refer to the minimal example above to see this paradigm in action.

options (*optional dict*) – Additional information to specify how the environment is reset (optional, depending on the specific environment)

RETURNS:

observation (*object*) – Observation of the initial state. This will be an element of observation_space (typically a numpy array) and is analogous to the observation returned by step().

info (*dictionary*) – This dictionary contains auxiliary information complementing observation. It should be analogous to the info returned by step().

gym.Env.render(self) → RenderFrame | List[RenderFrame] | None

Compute the render frames as specified by render_mode attribute during initialization of the environment.

The set of supported modes varies per environment. (And some third-party environments may not support rendering at all.) By convention, if render_mode is:

None (default): no render is computed.

human: render return None. The environment is continuously rendered in the current display or terminal. Usually for human consumption.

rgb_array: return a single frame representing the current state of the environment. A frame is a numpy.ndarray with shape (x, y, 3) representing RGB values for an x-by-y pixel image.

rgb_array_list: return a list of frames representing the states of the environment since the last reset. Each frame is a numpy.ndarray with shape (x, y, 3), as with rgb_array.

ansi: Return a strings (str) or StringIO.StringIO containing a terminal-style text representation for each time step. The text can include newlines and ANSI escape sequences (e.g. for colors).

2. Attributes

Env.action_space: Space[ActType]

This attribute gives the format of valid actions. It is of datatype *Space* provided by Gym. For example, if the action space is of type *Discrete* and gives the value *Discrete(2)*, this means there are two valid discrete actions: 0 & 1.

```
>>>env.action_space
Discrete(2)
>>>env.observation_space
Box(-3.4028234663852886e+38, 3.4028234663852886e+38, (4,), float32)
```

Env.observation_space: Space[ObsType]

This attribute gives the format of valid observations. It is of datatype Space provided by Gym. For example, if the observation space is of type Box and the shape of the object is (4,), this denotes a valid observation will be an array of 4 numbers. We can check the box bounds as well with attributes.

```
>>> env.observation_space.high
array([4.8000002e+00, 3.4028235e+38, 4.1887903e-01, 3.4028235e+38], dtype=float32)
>>>env.observation_space.low
array([-4.8000002e+00, -3.4028235e+38, -4.1887903e-01, -3.4028235e+38], dtype=float32)
```

Env.reward_range = (-inf, inf)
This attribute is a tuple corresponding to min and max possible rewards. Default range is set to (-inf,+inf). You can set it if you want a narrower range.

3. Additional Methods

gym.Env.close(self)

Override close in your subclass to perform any necessary cleanup.

Environments will automatically close() themselves when garbage collected or when the program exits.

Deep Reinforcement Learning Lab1: FrozenLake: A Slippery Challenge

https://github.com/badrhr/DRL-Lab-3-FrozenLake/

FrozenLake is a classic reinforcement learning environment provided by the OpenAI Gym library. It simulates the task of navigating a treacherous, icy terrain. The agent's goal is to reach the goal state without falling into any holes.

Key Characteristics:

- Grid World: The environment is represented as a 4x4 grid, with each cell representing a different state.
- Actions: The agent can take four actions: up, down, left, or right.
- State Transitions: Due to the slippery nature of the ice, the agent's actions may not always have the desired effect.
- Rewards: The agent receives a reward of 1 upon reaching the goal state and a reward of 0 otherwise.
- Terminal States: The goal state and hole states are terminal states.

FrozenLake is a popular choice for introducing reinforcement learning concepts due to its simplicity and the challenge it presents. It's a great environment to experiment with various reinforcement learning algorithms, including Q-learning.

For optimal performance and to avoid potential conflicts with system-wide Python installations, it is recommended to create a virtual environment using Anaconda. This will provide a dedicated Python environment for your project.

Lab Requirements:

- Software:
  - Python: Version 3.7.*
- Libraries:
  - gym==0.25.2
  - tensorflow==2.1.0
  - keras-rl2
  - numpy
  - pyglet
  - protobuf==3.20.*
  - pygame

https://github.com/badrhr/DRL-Lab-3-FrozenLake/tree/main

```python
import numpy as np
import gym

"""Task 2: Initialize the Environment"""

env = gym.make('FrozenLake-v1')

"""Task 3: Define Parameters"""

epsilon = 0.9  # Exploration rate
total_episodes = 100  # Number of episodes
max_steps = 100  # Maximum steps per episode
alpha = 0.85  # Learning rate
gamma = 0.95  # Discount factor

"""Task 4: Initialize the Q-table"""

Q = np.zeros((env.observation_space.n, env.action_space.n))

"""Task 5: Implement the `choose_action` Function"""

def choose_action(state):
    if np.random.uniform(0, 1) < epsilon:
        return env.action_space.sample()  # Explore randomly
    else:
        return np.argmax(Q[state, :])  # Exploit the best known action

"""  Task 6: Implement the `update` Function  """

def update(state, state2, reward, action, action2):
    predict = Q[state, action]
    target = reward + gamma * Q[state2, action2]
    Q[state, action] = Q[state, action] + alpha * (target - predict)

"""  Task 7: Implement the Training Loop  """

for episode in range(total_episodes):
    state = env.reset()
    action = choose_action(state)

    for t in range(max_steps):
        # Visualize the training
        env.render()
```

```
        # Take a step in the environment
        next_state, reward, done, _ = env.step(action)

        # Choose the next action
        next_action = choose_action(next_state)

        # Update the Q-table
        update(state, next_state, reward, action, next_action)

        state = next_state
        action = next_action

        if done:
            break

"""  Task 8: Evaluate the Performance  """

# Evaluate the performance
total_reward = 0
for _ in range(100):
    state = env.reset()
    while True:
        action = np.argmax(Q[state, :])
        next_state, reward, done, _ = env.step(action)
        total_reward += reward
        state = next_state
        if done:
            break
print("Average Reward:", total_reward / 100)

"""  Additional Task: Visualize the Q-table  """

print(Q)
```